

Unidad V

El Proceso de Diseño



El Proceso de Diseño

- **Diseño:** es el proceso creativo de transformación del problema en una solución.
- A la descripción de una solución también se denomina diseño.
- Se debe tener en cuenta que se pueden elaborar muchos diseños, que dan diferentes soluciones al problema.

El Proceso de Diseño

- **Diseño:** proceso de aplicar distintas técnicas y principios con el propósito de definir un producto con los suficientes detalles como para permitir su realización física.
- Con el diseño se pretende construir un sistema que:
 - Satisfaga determinada especificación del mismo.
 - Se ajuste a las limitaciones impuestas por el medio destino.
 - Respete requisitos sobre forma, rendimiento, utilización de recursos, coste, etc.

El diseño de SW es un proceso iterativo a través del cual se traducen los requisitos en un representación del SW.

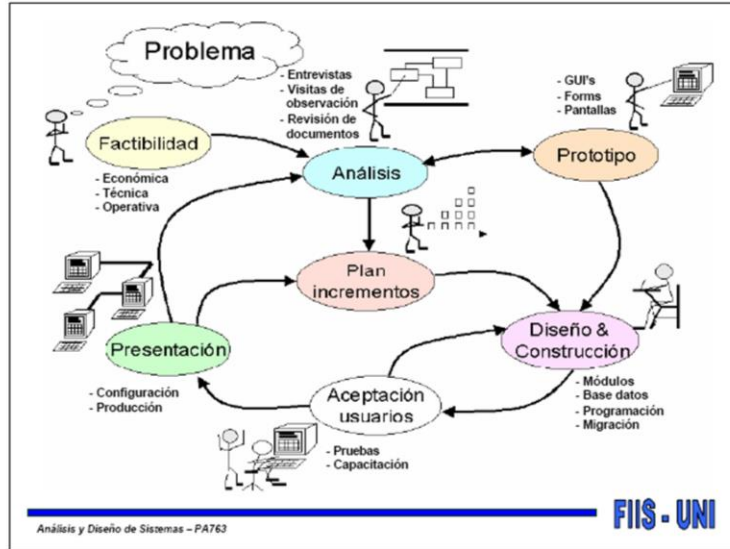
El Proceso de Diseño

- El **Diseño** debe ser una guía que puedan leer y entender los que construyan el código y los que prueban y mantienen el software.

Diseñar un sistema es determinar un conjunto de componentes y de interfaces entre ellos que satisfagan un conjunto específico de requerimientos.

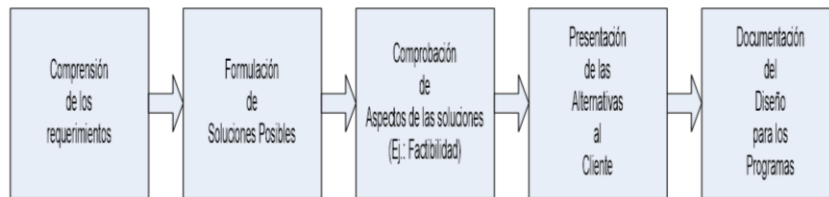
El diseño de SW es un proceso iterativo a través del cual se traducen los requisitos en una representación del SW.

El Proceso de Diseño



El proceso de desarrollo de software es aquel en el que las necesidades del usuario son traducidas en requisitos de software, estos transformados en diseño y el diseño implementado en código.

El Proceso de Diseño



El Proceso de Diseño

Primero se elabora un Diseño conceptual, el cual **le dice al cliente, exactamente, qué hará el sistema.**

- Se describe en el lenguaje del cliente.
- No contiene expresiones en jerga técnica.
- Describe claramente las funciones del sistema.
- Es independiente de la implementación.
- Está vinculado con los documentos sobre requerimientos.

El Proceso de Diseño

Es decir, describe los aspectos del sistema respondiendo a cuestiones como:

- ¿De dónde provienen los datos?
- ¿Qué les sucede a los datos en el interior del sistema?
- ¿Qué elecciones se ofrecerán a los usuarios?
- ¿Qué aspecto tendrán los informes y pantallas?
- Al cliente no se le dice cómo se almacenarán los datos o qué clase de sistema de administración de BBDD manipulará los datos. Es decir, el protocolo y la topología de red no se indican en este nivel

El Proceso de Diseño

Una vez que el cliente aprueba el diseño conceptual, se vuelve el diseño en un documento mucho más detallado: el **Diseño técnico**, el cual describe la configuración del hardware, las necesidades de software, las interfaces de comunicación, las E/S del sistema, la arquitectura de red, etc.

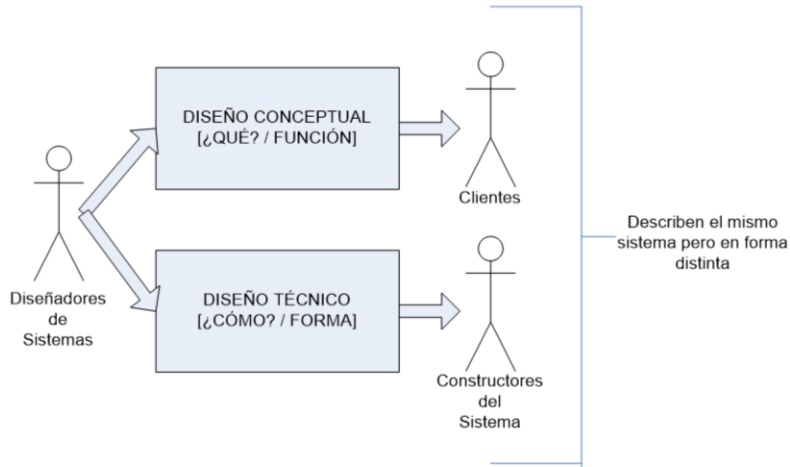
Es decir describe la jerarquía y funciones de los componentes, la estructura de datos y el flujo de los datos.

El Proceso de Diseño

Las características de un buen diseño técnico son:

- Describe los principales componentes de hardware y sus funciones.
- Incluye la jerarquía y funciones de los componentes de software.
- Incluye las estructuras de datos y el flujo de datos.

El Proceso de Diseño



- En ocasiones, los clientes están muy preparados y pueden entender el qué y el cómo en forma conjunta, y en esos casos, los diseños conceptual y técnico pueden combinarse en un único documento de diseño.
- Es importante vincular los diseños conceptual y técnico, de modo que los cambios en uno de ellos se reflejen como cambios en el otro.
- Un sistema queda definido por sus límites, atributos, entidades y relaciones.

El Proceso de Diseño

El Proceso de Diseño

De lo general a lo particular:

1. El proceso comienza con un enfoque en la arquitectura.
2. Se definen los subsistemas.
3. Se establecen los mecanismos de comunicación entre los subsistemas.
4. Se identifican los componentes.
5. Se desarrolla una descripción detallada de cada componente.

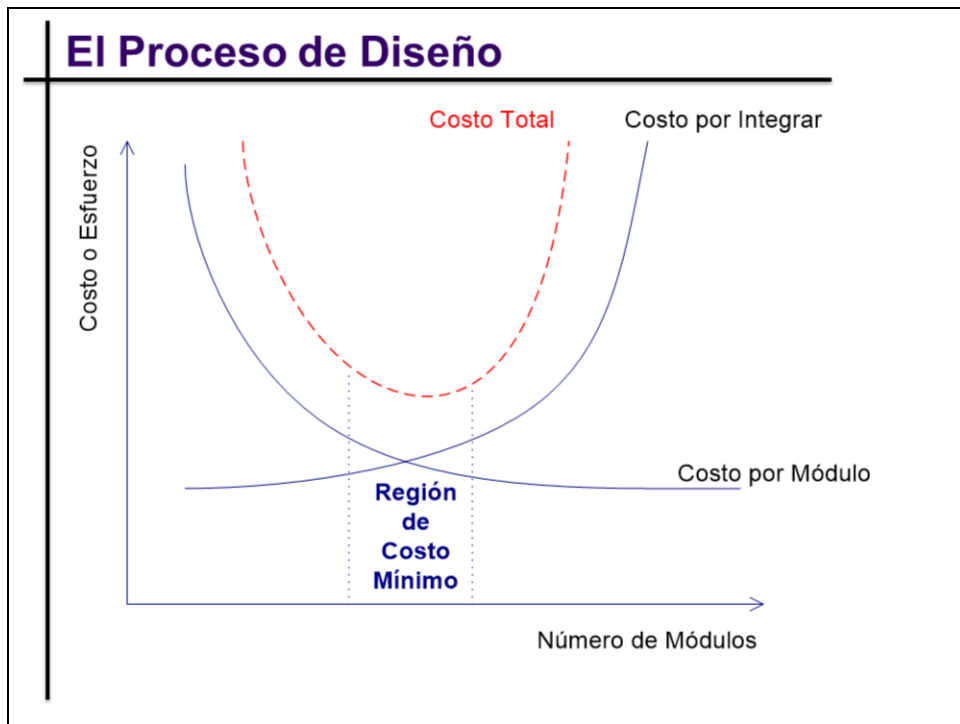
El proceso de diseño avanza de una visión general de software a una visión más estrecha que define el detalle requerido para implementar un sistema. Los pasos de dicho proceso se pueden enumerar de la siguiente manera:

1. El proceso comienza con un enfoque en la arquitectura.
2. Se definen los subsistemas.
3. Se establecen los mecanismos de comunicación entre los subsistemas.
4. Se identifican los componentes.
5. Se desarrolla una descripción detallada de cada componente.

Además, se diseñan las interfaces internas, externa y del usuario.

El Proceso de Diseño

La **modularidad** es una de las características de un buen diseño, con lo que los componentes tendrán entradas y salidas claramente definidas y cada componente tiene un propósito claramente establecido.



Luego M sería el número de módulos que resultaría en un costo de desarrollo mínimo, pero, no se tiene la sofisticación necesaria para predecir M con seguridad.

Además, la modularidad permite aislar aquellas partes del problema que son difíciles de tratar, y los niveles de abstracción permiten comprender el problema en niveles crecientes de detalle.

El Proceso de Diseño

El **diseño modular efectivo** reduce la complejidad, facilita los cambios y produce como resultado una implementación más sencilla, permitiendo el desarrollo paralelo de las diferentes partes del sistema.

El Proceso de Diseño

Otras características de un buen Diseño

1. Facilidad de comprender (Legibilidad)
2. Facilidad de implementar
3. Facilidad de probar
4. Facilidad de modificar (Extensibilidad)
5. Correcta traducción de las especificaciones de requerimientos (Funcionalidad)
6. Facilidad de uso
7. Eficiencia
8. Reutilización
9. Robustez
10. Compatibilidad
11. Portabilidad

Dan como resultado productos de calidad. Por calidad se entiende a la adecuación del software a los requisitos exigidos.

Extensibilidad. Facilidad para adaptarse a los cambios en las especificaciones. Se logra haciendo simples los diseños de los módulos autónomos.

Funcionalidad. Conjunto de posibilidades ofrecido por un sistema.

Solidez o robustez. Habilidad para funcionar aún en condiciones anormales, es decir, con aquellos casos no explicitados en las especificaciones. Si se presentan el sistema termina “limpiamente”.

Facilidad de uso. Facilidad con la que personas con diferentes niveles de experiencia pueden aprender a usar los productos software.

Eficiencia. Facilidad de utilizar el mínimo de recursos de cómputo para conseguir mayor rapidez y menor necesidad de almacenamiento.

Reutilización. Habilidad para utilizar de nuevo productos de software completos o partes de ellos en nuevas aplicaciones.

Compatibilidad. Facilidad con la que un producto software puede combinarse con otros. Se logra homogeneidad en el diseño y estandarización en la comunicación entre programas.

Portabilidad. Facilidad de transferir productos a diferentes plataformas.

El Proceso de Diseño

Independencia de los componentes: para reconocer y medir el grado de independencia de los componentes de un diseño se aplican dos conceptos:

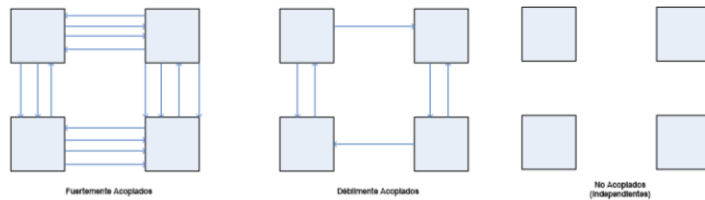
- Acoplamiento
- Cohesión

La **independencia funcional** se adquiere desarrollando módulos con una clara función, evitando una excesiva interacción con otros módulos.

El Proceso de Diseño

Acoplamiento

Dos componentes están altamente acoplados cuando existe mucha dependencia entre ellos. Los poco acoplados tienen alguna dependencia, pero las interconexiones entre ellos son débiles. Los componentes no acoplados no tienen interconexiones con el resto; son completamente independientes)



El **acoplamiento** es la medida de la interconexión entre módulos de un programa.

El acoplamiento depende de varios aspectos:

- Las referencias hechas de un componente a otro.
- La cantidad de datos pasados de un componente a otro.
- El grado de control que un componente tiene sobre el otro.
- El grado de complejidad de la interfaz entre los componentes

El rango de medidas de Acoplamiento es:

- Acoplamiento de contenido: Un componente modifica a otro. El componente modificado es completamente dependiente del que lo modifica.
- Acoplamiento común: Por que ambos componentes utilizan un área común para almacenar datos.
- Acoplamiento por control: Un componente pasa parámetros para controlar la actividad del otro componente.
- Acoplamiento por estampado: Se usa una estructura de

datos para pasar información de un componente a otro.

- Acoplamiento por datos: Solo se pasan datos de un componente a otro.
- No Acoplados.

El Proceso de Diseño

Cohesión

Se refiere al grado de “adhesivo” interno con el que se ha construido el componente.

Cuando más cohesivo es un componente, más relacionadas están las partes internas de él, tanto entre ellas como en relación al propósito global.

Un componente es cohesivo si todos sus elementos están orientados a la realización de una única tarea y son esenciales para llevarla a cabo, sino serían dos módulos.

Un **módulo cohesivo** ejecuta una tarea sencilla de un procedimiento de SW y requiere poca interacción con procedimientos que ejecutan otras partes de un programa.

Los grados de Cohesión son:

- Cohesión coincidental: Ocurre cuando las partes de un componente no tienen relación alguna entre sí y se encuentran en un mismo componente por razones de conveniencia o simplemente porque sí.
- Cohesión lógica: Algunas funciones o elementos de datos relacionados lógicamente están puestos en el mismo componente. Estos elementos no están relacionados funcionalmente.
- Cohesión temporal: Las funciones en sí sólo están relacionadas por el momento en que ocurren.
- Cohesión procedimental: Las funciones se agrupan en un mismo componente para asegurar el orden.
- Cohesión comunicativa: Las funciones operan, o producen, el mismo conjunto de datos.

- Cohesión secuencial: La salida proporcionada por una parte del componente actúa como entrada para la parte que le sigue.
- Cohesión funcional: Cada elemento de proceso es esencial para la realización de una única función y todos los elementos esenciales están contenidos en un único componente. El componente no sólo realiza la función para la cual ha sido diseñado, sino que realiza esa función y nada más.

El Proceso de Diseño

**Interesa una cohesión alta y
un acoplamiento bajo**

La **independencia funcional** se adquiere desarrollando módulos con una clara función, evitando una excesiva interacción con otros módulos.

El Proceso de Diseño

El diseño debe ser realizado bajo el concepto de **tolerancia a fallas**.

Luego, cuando ocurren excepciones (Son situaciones contrarias a lo que se espera que normalmente haga el sistema, como por ejemplo: fracaso al proporcionar un servicio, proporcionar un servicio o dato erróneo o corrupción de datos) se tienen varios criterios para manejarla:

Ante una falla (desvío del sistema respecto de su comportamiento requerido) las posibles soluciones son:

Eliminarlo

prevenir las condiciones de entrada

agregar comportamiento para la recuperación del daño provocado por la falla

El Proceso de Diseño

El diseño debe ser realizado bajo el concepto de **tolerancia a fallas**.

- Corregir: Se vuelve al estado previo y se reintenta con una estrategia distinta.
- Reintentar: Se vuelve al estado previo y se reintenta con la misma estrategia.
- Informar: Se vuelve al estado previo y se informa sin proporcionar el servicio.

Ante una falla (desvío del sistema respecto de su comportamiento requerido) las posibles soluciones son:

Eliminarlo

prevenir las condiciones de entrada

agregar comportamiento para la recuperación del daño provocado por la falla

El Proceso de Diseño

Además, es muy importante considerar desde el inicio del diseño los siguientes parámetros:

- Facilidad de uso.
- Confiabilidad (frecuencia y severidad de las fallas).
- Desempeño (velocidad de procesamiento, tiempo de respuesta, rendimiento y eficacia).
- Facilidad de mantenimiento.
- Seguridad

Debe tenerse en cuenta que no todos los parámetros tienen el mismo peso.

El Proceso de Diseño

Diferentes estrategias de diseño:

Diseño a nivel de componentes

Componente: Es una parte modular, desplegable y reemplazable de un sistema que encapsula implementación y expone un conjunto de interfaces.

Es un bloque de construcción modular para el software de cómputo.

Diseño a nivel de componentes: Define las estructuras de datos, los algoritmos, las características de la interfaz y los mecanismos de comunicación asignados a cada componente de software. Como resultado se obtiene el diseño de cada componente, representado en una notación gráfica, tabular o textual.

- **Componente:** Es una parte modular, desplegable y reemplazable de un sistema que encapsula implementación y expone un conjunto de interfaces. Es un bloque de construcción modular para el software de cómputo.

Concepto orientado a objetos.

- **Componente:** Es un conjunto de clases que colaboran entre sí. Cada clase un componente se ha elaborado completamente para incluir todos los atributos y las operaciones relevantes para su implementación. Como parte de la elaboración del diseño, también deben definirse todas las interfaces (mensajes) que permiten que las clases se comuniquen con otras clases de diseño.

Concepto convencional.

- **Componente:** Es un elemento funcional de un programa que incorpora la lógica del procesamiento, las estructuras interna

de los datos necesarios para implementar dicha lógica, y una interfaz que permita la invocación del componente y el paso de los datos. También llamado módulo.

El Proceso de Diseño

Diferentes estrategias de diseño:

Concepto orientado a objetos.

Componente: Es un conjunto de clases que colaboran entre sí. Cada clase un componente se ha elaborado completamente para incluir todos los atributos y las operaciones relevantes para su implementación.

Como parte de la elaboración del diseño, también deben definirse todas las interfaces (mensajes) que permiten que las clases se comuniquen con otras clases de diseño.

Diseño a nivel de componentes: Define las estructuras de datos, los algoritmos, las características de la interfaz y los mecanismos de comunicación asignados a cada componente de software. Como resultado se obtiene el diseño de cada componente, representado en una notación gráfica, tabular o textual.

- **Componente:** Es una parte modular, desplegable y reemplazable de un sistema que encapsula implementación y expone un conjunto de interfaces. Es un bloque de construcción modular para el software de cómputo.

Concepto orientado a objetos.

- **Componente:** Es un conjunto de clases que colaboran entre sí. Cada clase un componente se ha elaborado completamente para incluir todos los atributos y las operaciones relevantes para su implementación. Como parte de la elaboración del diseño, también deben definirse todas las interfaces (mensajes) que permiten que las clases se comuniquen con otras clases de diseño.

Concepto convencional.

- **Componente:** Es un elemento funcional de un programa que incorpora la lógica del procesamiento, las estructuras interna

de los datos necesarios para implementar dicha lógica, y una interfaz que permita la invocación del componente y el paso de los datos. También llamado módulo.

El Proceso de Diseño

Diferentes estrategias de diseño:

Diseño a nivel de componentes

- **Concepto convencional.**

Componente: Es un elemento funcional de un programa que incorpora la lógica del procesamiento, las estructuras interna de los datos necesarios para implementar dicha lógica, y una interfaz que permita la invocación del componente y el paso de los datos. También llamado módulo.

Diseño a nivel de componentes: Define las estructuras de datos, los algoritmos, las características de la interfaz y los mecanismos de comunicación asignados a cada componente de software. Como resultado se obtiene el diseño de cada componente, representado en una notación gráfica, tabular o textual.

- **Componente:** Es una parte modular, desplegable y reemplazable de un sistema que encapsula implementación y expone un conjunto de interfaces. Es un bloque de construcción modular para el software de cómputo.

Concepto orientado a objetos.

- **Componente:** Es un conjunto de clases que colaboran entre sí. Cada clase un componente se ha elaborado completamente para incluir todos los atributos y las operaciones relevantes para su implementación. Como parte de la elaboración del diseño, también deben definirse todas las interfaces (mensajes) que permiten que las clases se comuniquen con otras clases de diseño.

Concepto convencional.

- **Componente:** Es un elemento funcional de un programa que incorpora la lógica del procesamiento, las estructuras interna

de los datos necesarios para implementar dicha lógica, y una interfaz que permita la invocación del componente y el paso de los datos. También llamado módulo.

El Proceso de Diseño

Diseño estructurado

Los programas se dividen en módulos definidos por las funciones (subrutinas, procedimientos, funciones, etc.).

Es la manera natural de organización de una ingeniería industrial, donde se divide al proyecto en tareas y los productos pasan de una tarea a otra en forma secuencial.

Este método da lugar a enormes dificultades cuando se debe modificar o extender el programa ya que los datos son compartidos por todos los módulos funcionales.

El Proceso de Diseño

Diseño OO

Los elementos fundamentales son los datos. Las funciones (los métodos) se definen e implementan de acuerdo a la modularización establecida por los datos.

Cada módulo del programa (objeto) es responsable de sus propios datos, los cuales solo pueden ser modificados por los métodos propios y “observados” por los otros objetos a través de protocolos que resguardan el encapsulamiento.

El universo computacional está poblado por objetos, cada uno responsable de sí mismo, y comunicándose con los demás por medio de mensajes. Cada objeto representa una instancia de alguna clase, y estas clases son miembros de una jerarquía de clase unidas vía relaciones de herencia.

La diferencia entre un objeto y una clases que un objeto es una entidad concreta que existe en tiempo y espacio, mientras que una clase representa una abstracción, la esencia de un objeto.

Robustez. La robustez de un sistema debe ser uno de los objetivos principales del diseño. El sistema debe ser protegidos contra errores y ofrecer diagnósticos que permitan identificar fallas, en particular aquellas que son fatales. Durante el desarrollo, a veces es bueno insertar instrucciones internas en el código para descubrir fallas, aunque luego se eliminan durante la producción. En general, se debe escoger lenguajes de programación que apoyen estos aspectos, como son el manejo de excepciones. El encapsulamiento es fundamental para la robustez del sistema. Ocultar la información interna, atributos e implementación de métodos de una clase, permite cambiarla sin afectar al resto del sistema. Únicamente la interfaz de los métodos afecta a las demás clases.

Reuso. El reuso es un aspecto fundamental del diseño. Cuanto más se pueda reutilizar el código será mejor la robustez del sistema. El encapsulamiento es muy efectivo para lograr el reuso, se aplica tanto al nivel de los objetos como de componentes desarrollados en otras aplicaciones. Estos componentes se reutilizan como se diseñaron, agregándolos a nuevas interfaces.

Extensibilidad. La mayor parte de los sistemas son extendidos de manera imprevista. Se debe encapsular otra vez las clases, ocultando su estructura interna a las otras clases. Sólo los métodos de las clases deben acceder sus atributos.

El Proceso de Diseño

Diseño OO

Otros conceptos que ayudan a la extensibilidad y modificación futura son: abstracción, herencia y polimorfismo.

Clase: Es un patrón o plantilla para una categoría de objetos con la misma estructura. Es una fábrica de instancia.

El universo computacional está poblado por objetos, cada uno responsable de sí mismo, y comunicándose con los demás por medio de mensajes. Cada objeto representa una instancia de alguna clase, y estas clases son miembros de una jerarquía de clase unidas vía relaciones de herencia.

La diferencia entre un objeto y una clase es que un objeto es una entidad concreta que existe en tiempo y espacio, mientras que una clase representa una abstracción, la esencia de un objeto.

Robustez. La robustez de un sistema debe ser uno de los objetivos principales del diseño. El sistema debe ser protegido contra errores y ofrecer diagnósticos que permitan identificar fallas, en particular aquellas que son fatales. Durante el desarrollo, a veces es bueno insertar instrucciones internas en el código para descubrir fallas, aunque luego se eliminan durante la producción. En general, se debe escoger lenguajes de programación que apoyen estos aspectos, como son el manejo de excepciones. El encapsulamiento es fundamental para la robustez del sistema. Ocultar la información interna, atributos e implementación de métodos de una clase, permite cambiarla sin afectar al resto del sistema. Únicamente la interfaz de los métodos afecta a las demás clases.

Reuso. El reuso es un aspecto fundamental del diseño. Cuanto más se pueda reutilizar el código será mejor la robustez del sistema. El encapsulamiento es muy efectivo para lograr el reuso, se aplica tanto al nivel de los objetos como de componentes desarrollados en otras aplicaciones. Estos componentes se reutilizan como se diseñaron, agregándolos a nuevas interfaces.

Extensibilidad. La mayor parte de los sistemas son extendidos de manera imprevista. Se debe encapsular otra vez las clases, ocultando su estructura interna a las otras clases. Sólo los métodos de las clases deben acceder sus atributos.

El Proceso de Diseño

Patrones de Diseño

Los patrones de diseño son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software.

Un patrón de diseño resulta ser una solución a un problema de diseño.

Para que una solución sea considerada un patrón debe poseer ciertas características:

- **efectividad**
- **reutilizable**

Los patrones de diseño son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño de interacción o interfaces.

Un patrón de diseño resulta ser una solución a un problema de diseño. Para que una solución sea considerada un patrón debe poseer ciertas características. Una de ellas es que debe haber comprobado su **efectividad** resolviendo problemas similares en ocasiones anteriores. Otra es que debe ser **reutilizable**, lo que significa que es aplicable a diferentes problemas de diseño en distintas circunstancias.

"Cada patrón describe un problema que ocurre infinidad de veces en nuestro entorno, así como la solución al mismo, de tal modo que podemos utilizar esta solución un millón de veces más adelante sin tener que volver a pensarla otra vez."

El Proceso de Diseño

Objetivos de los Patrones

Los patrones de diseño pretenden:

- Proporcionar catálogos de elementos reusables.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño.
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores.

Los patrones de diseño pretenden:

- Proporcionar catálogos de elementos reusables en el diseño de sistemas software.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño.
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Asimismo, no pretenden:

- Imponer ciertas alternativas de diseño frente a otras.
- Eliminar la creatividad inherente al proceso de diseño.

No es obligatorio utilizar los patrones, solo es aconsejable en el caso de tener el mismo problema o similar que soluciona el patrón, siempre teniendo en cuenta que en un caso particular puede no ser aplicable. "Abusar o forzar el uso de los patrones puede ser un error".

El Proceso de Diseño

Objetivos de los Patrones

Objetivos

Asimismo, no pretenden:

- Imponer ciertas alternativas de diseño frente a otras.
- Eliminar la creatividad inherente al proceso de diseño.

No es obligatorio utilizar los patrones, solo es aconsejable en el caso de tener el mismo problema o similar que soluciona el patrón.

Los patrones de diseño pretenden:

- Proporcionar catálogos de elementos reusables en el diseño de sistemas software.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño.
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Asimismo, no pretenden:

- Imponer ciertas alternativas de diseño frente a otras.
- Eliminar la creatividad inherente al proceso de diseño.

No es obligatorio utilizar los patrones, solo es aconsejable en el caso de tener el mismo problema o similar que soluciona el patrón, siempre teniendo en cuenta que en un caso particular puede no ser aplicable. "Abusar o forzar el uso de los patrones puede ser un error".

El Proceso de Diseño

Principales Patrones

Patrones de creación

- Object Pool: se obtienen objetos nuevos a través de la clonación. El proceso de clonación se inicia instanciando un tipo de objeto de la clase que queremos clonar.
- Factory Method (método de fabricación): centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la diversidad de casos particulares que se pueden prever, para elegir el subtipo que crear.
- Prototype (prototipo): crea nuevos objetos clonándolos de una instancia ya existente.
- Singleton (instancia única): garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia.

Patrones de creación

- Object Pool: se obtienen objetos nuevos a través de la clonación. Utilizado cuando el costo de crear una clase es mayor que el de clonarla. Especialmente con objetos muy complejos. Se especifica un tipo de objeto a crear y se utiliza una interfaz del prototipo para crear un nuevo objeto por clonación. El proceso de clonación se inicia instanciando un tipo de objeto de la clase que queremos clonar.
- Abstract Factory (fábrica abstracta): permite trabajar con objetos de distintas familias de manera que las familias no se mezclen entre sí y haciendo transparente el tipo de familia concreta que se esté usando.
- Builder (constructor virtual): abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto.
- Factory Method (método de fabricación): centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la casuística, es decir, la diversidad de casos particulares que se pueden prever, para elegir el subtipo que crear.
- Prototype (prototipo): crea nuevos objetos clonándolos de una instancia ya existente.
- Singleton (instancia única): garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia.

[<http://patronesdediseno.net16.net/creacion.html>]

El Proceso de Diseño

Principales Patrones

Patrones estructurales

- Adapter (Adaptador): Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla.
- Composite (Objeto compuesto): Permite tratar objetos compuestos como si de uno simple se tratase.
- Decorator (Envoltorio): Añade funcionalidad a una clase dinámicamente.
- Facade (Fachada): Provee de una interfaz unificada simple para acceder a una interfaz o grupo de interfaces de un subsistema.
- Flyweight (Peso ligero): Reduce la redundancia cuando gran cantidad de objetos poseen idéntica información.
- Proxy: Mantiene un representante de un objeto.

Patrones estructurales

- Adapter (Adaptador): Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla.
- Bridge (Puente): Desacopla una abstracción de su implementación.
- Composite (Objeto compuesto): Permite tratar objetos compuestos como si de uno simple se tratase.
- Decorator (Envoltorio): Añade funcionalidad a una clase dinámicamente.
- Facade (Fachada): Provee de una interfaz unificada simple para acceder a una interfaz o grupo de interfaces de un subsistema.
- Flyweight (Peso ligero): Reduce la redundancia cuando gran cantidad de objetos poseen idéntica información.
- Proxy: Mantiene un representante de un objeto.
- Módulo: Agrupa varios elementos relacionados, como clases, singletons, y métodos, utilizados globalmente, en una entidad única.

[<http://patronesdediseno.net16.net/estructurales.html>]

El Proceso de Diseño

Principales Patrones

Patrones de comportamiento

- Interpreter (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.
- Mediator (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.
- Memento (Recuerdo): Permite volver a estados anteriores del sistema.
- Observer (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.

Patrones de comportamiento

- Chain of Responsibility (Cadena de responsabilidad): Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.
- Command (Orden): Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.
- Interpreter (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.
- Iterator (Iterador): Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de estos.
- Mediator (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.
- Memento (Recuerdo): Permite volver a estados anteriores del sistema.
- Observer (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.
- State (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.
- Strategy (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.
- Template Method (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.
- Visitor (Visitante): Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

[<http://patronesdediseno.net16.net/comportamiento.html>]

El Proceso de Diseño

Principales Patrones

Patrones de comportamiento

- State (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.
- Strategy (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.
- Template Method (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.

Patrones de comportamiento

- Chain of Responsibility (Cadena de responsabilidad): Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.
- Command (Orden): Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.
- Interpreter (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.
- Iterator (Iterador): Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de estos.
- Mediator (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.
- Memento (Recuerdo): Permite volver a estados anteriores del sistema.
- Observer (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.
- State (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.
- Strategy (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.
- Template Method (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.
- Visitor (Visitante): Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

[<http://patronesdediseno.net16.net/comportamiento.html>]

El Proceso de Diseño

Patrones de Arquitectura

Los patrones de arquitectura ofrecen soluciones a problemas de arquitectura de software en ingeniería de software.

Dan una descripción de los elementos y el tipo de relación que tienen junto con un conjunto de restricciones sobre cómo pueden ser usados, es decir, expresa un esquema de organización estructural esencial para un sistema de software, que consta de subsistemas, sus responsabilidades e interrelaciones.

En comparación con los patrones de diseño, los patrones arquitectónicos tienen una nivel de abstracción mayor.

Los **patrones arquitectónicos**, o **patrones de arquitectura**, son patrones de diseño de software que ofrecen soluciones a problemas de arquitectura de software en ingeniería de software. Dan una descripción de los elementos y el tipo de relación que tienen junto con un conjunto de restricciones sobre cómo pueden ser usados. Un patrón arquitectónico expresa un esquema de organización estructural esencial para un sistema de software, que consta de subsistemas, sus responsabilidades e interrelaciones. En comparación con los patrones de diseño, los patrones arquitectónicos tienen una nivel de abstracción mayor.

El Proceso de Diseño

Patrones de Arquitectura

Un patrón arquitectónico no es una arquitectura como tal, sino es más un concepto que captura elementos esenciales de una arquitectura de software.

Muchas arquitecturas diferentes pueden implementar el mismo patrón y por lo tanto compartir las mismas características.

Uno de los aspectos más importantes de los patrones arquitectónicos es que encarnan diferentes atributos de calidad. Por ejemplo, algunos patrones representan soluciones a problemas de rendimiento y otros pueden ser utilizados con éxito en sistemas de alta disponibilidad.

Aunque un patrón arquitectónico comunica una imagen de un sistema, no es una arquitectura como tal. Un patrón arquitectónico es más un concepto que captura elementos esenciales de una arquitectura de software. Muchas arquitecturas diferentes pueden implementar el mismo patrón y por lo tanto compartir las mismas características. Además, los patrones son a menudo definidos como una cosa "estrictamente descrita y comúnmente disponible". Por ejemplo, la arquitectura en capas es un estilo de llamamiento-y-regreso, cuando define uno un estilo general para interaccionar. Cuando esto es descrito estrictamente y comúnmente disponible, es un patrón.

Uno de los aspectos más importantes de los patrones arquitectónicos es que encarnan diferentes atributos de calidad. Por ejemplo, algunos patrones representan soluciones a problemas de rendimiento y otros pueden ser utilizados con éxito en sistemas de alta disponibilidad. En las primeras fases del diseño, un arquitecto de software escoge qué patrones arquitectónicos mejor ofrecen las calidades deseadas para el sistema.

Algunos de esos aspectos no funcionales son:

- Control de acceso: Hay muchas situaciones en las cuales el acceso a

datos, características y funcionalidad son limitadas a la definición de los usuarios. Desde un punto de vista arquitectónico, acceder a determinadas partes del software debe tener un riguroso control.

- Concurrencia: Muchas aplicaciones deben manejar múltiples tareas de forma que simule el paralelismo. Hay muchas formas de manejar esta concurrencia, y cada una puede ser presentada por un patrón arquitectónico diferente.
- Distribución: El problema de distribución dirige el problema de forma en que los sistemas o componentes se comunican con otros en un entorno distribuido. El patrón más común para afrontar el problema es "the broker". Actuando como un "middleman" entre el componente cliente y el servidor. El cliente envía un mensaje al "broker" y éste se encarga de completar la conexión.

Persistencia:

Los datos persistentes son almacenados en bases de datos o archivos y pueden ser leídos o modificados por otros procesos más adelante. En los entornos orientados a objetos esto va más allá, y lo que puede ser accedido o modificable son las propiedades de los objetos.

El Proceso de Diseño

Patrones de Arquitectura

Ejemplos de patrones arquitectónicos:

- Programación por capas
- Tres niveles
- Pipeline
- Invocación implícita
- Arquitectura en pizarra
- Arquitectura dirigida por eventos
- Peer-to-peer
- Arquitectura orientada a servicios
- Modelo-Vista-Controlador

Ejemplos de patrones arquitectónicos:

- Programación por capas
- Tres niveles
- Pipeline
- Invocación implícita
- Arquitectura en pizarra
- Arquitectura dirigida por eventos
- Peer-to-peer
- Arquitectura orientada a servicios
- Modelo-Vista-Controlador



FIN